Watermark and Garbage Collection

In this chapter, you will implement necessary structures to track the lowest read timestamp being used by the user, and collect unused versions from SSTs when doing the compaction.

To run test cases,

```
cargo x copy-test --week 3 --day 4
cargo x scheck
```

Task 1: Implement Watermark

In this task, you will need to modify:

```
src/mvcc/watermark.rs
```

Watermark is the structure to track the lowest `read_ts` in the system. When a new transaction is created, it should call `add_reader` to add its read timestamp for tracking. When a transaction aborts or commits, it should remove itself from the watermark. The watermark structures returns the lowest `read_ts` in the system when `watermark()` is called. If there are no ongoing transactions, it simply returns `None`.

You may implement watermark using a `BTreeMap`. It maintains a counter that how many snapshots are using this read timestamp for each `read_ts`. You should not have entries with 0 readers in the b-tree map.

Task 2: Maintain Watermark in Transactions

In this task, you will need to modify:

```
src/mvcc/txn.rs
src/mvcc.rs
```

You will need to add the `read_ts` to the watermark when a transaction starts, and remove it when `drop` is called for the transaction.

Task 3: Garbage Collection in Compaction

In this task, you will need to modify:

```
src/compact.rs
```

Now that we have a watermark for the system, we can clean up unused versions during the compaction process.

- If a version of a key is above watermark, keep it.
- For all versions of a key below or equal to the watermark, keep the latest version.

For example, if we have watermark=3 and the following data:

```
a@4=del <- above watermark
a@3=3   <- latest version below or equal to watermark
a@2=2   <- can be removed, no one will read it
a@1=1   <- can be removed, no one will read it
b@1=1   <- latest version below or equal to watermark
c@4=4   <- above watermark
d@3=del <- can be removed if compacting to bottom-most level
d@2=2   <- can be removed
```

If we do a compaction over these keys, we will get:

```
a@4=del
a@3=3
b@1=1
c@4=4
d@3=del (can be removed if compacting to bottom-most level)
```

Assume these are all keys in the engine. If we do a scan at ts=3, we will get `a=3,b=1,c=4` before/after compaction. If we do a scan at ts=4, we will get `b=1,c=4` before/after compaction. Compaction *will not* and *should not* affect transactions with read timestamp >= watermark.

Test Your Understanding

- In our implementation, we manage watermarks by ourselves with the lifecycle of `Transaction` (so-called un-managed mode). If the user intends to manage key timestamps and the watermarks by themselves (i.e., when they have their own timestamp generator), what do you need to do in the write_batch/get/scan API to validate their requests? Is there any architectural assumption we had that might be hard to maintain in this case?
- Why do we need to store an `Arc` of `Transaction` inside a transaction iterator?

- What is the condition to fully remove a key from the SST file?
- For now, we only remove a key when compacting to the bottom-most level. Is there any other prior time that we can remove the key? (Hint: you know the start/end key of each SST in all levels.)
- Consider the case that the user creates a long-running transaction and we could not garbage collect anything. The user keeps updating a single key. Eventually, there could be a key with thousands of versions in a single SST file. How would it affect performance, and how would you deal with it?

Bonus Tasks

- **O(1) Watermark.** You may implement an amortized O(1) watermark structure by using a hash map or a cyclic queue.